DOCUMENTACION 2942 SPACE INVASION:

p1. Objetivo y Alcance

El objetivo del sistema es desarrollar un **videojuego 2D tipo “space shooter”** en Processing, donde el jugador controla una nave espacial que debe eliminar enemigos y recoger potenciadores (*power-ups*) para aumentar su supervivencia y puntaje.

Alcance:

- Juego de un solo jugador.
- Combate en tiempo real con registro de puntaje y condición de *Game Over*.
- Diferentes tipos de enemigos y potenciadores (vida, escudo, ametralladora).
- Efectos visuales de explosión y fondo espacial animado.
- Tiene que tener un sistema de aumento de dificultad.
- Historial que salve la puntuación de los jugadores.
- Crear Nave Aliada con movimiento autónomo.

p2. Descripción de alto nivel del sistema

El sistema está diseñado bajo un modelo **orientado a objetos**, donde cada entidad del juego se encapsula en una clase independiente.
La coordinación general se realiza a través del `GameManager`, encargado del ciclo principal, la detección de colisiones y la gestión de estados.

Componentes principales:

- `GameManager`: Es una clase de acceso global. Controla el flujo del juego. Al actualizar, se encarga de dibujar entidades, actualizar estados y la detección de colisiones.

- `SoundManager`: Es una de las clases de acceso global. Controla los sonidos del juego. Es llamado de varias clases para reproducir sonidos.

- `DataManager`: Es una de las clases de acceso global. Se la llama para recolectar datos de la partida actual. Al finalizar la partida guarda los datos en un archivo .csv . Con estos datos, la clase también hace gráficos mostrando las distribuciones de las estadísticas al jugador.

- `Nave`: clase **abstracta base** que define el comportamiento común de las naves (posición, movimiento, vida).

- `NaveAliada`: clase que **representa al jugador**, heredando de `Nave` e implementando los controles del usuario, genera **`Balas Aliadas`** y usa las diferentes clases Power ups y es el encargado de activar sus efectos.

- `NaveEnemiga`: clase que **`hereda de Nave`**, define el comportamiento autónomo de los enemigos como el movimiento y generación de Balas Enemigas.

- `Bala`, `BalaAliada`, `BalaEnemiga`: jerarquía de proyectiles, con posiciones, velocidades y movimiento.

- `PowerUp`: Elementos coleccionables que otorgan mejoras temporales o permanentes. Al colisionar con la Nave Aliada, es consumido y activa eventos de la Nave Aliada.

- `FondoEspacial`: administra la animación del fondo para simular desplazamiento espacial.

- `Explosion`: maneja los efectos visuales de destrucción.

- `Game`: punto de entrada del programa, contiene las funciones `setup()` y `draw()` de Processing.

Ciclo principal:

1. Inicialización del entorno y carga de entidades.
2. Actualización del estado de cada objeto (naves, balas, power-ups, fondo).
3. Detección y gestión de colisiones.
4. Renderizado de todos los elementos visuales.
5. Control de estados, vidas, puntaje y condiciones de victoria o derrota.
6. Detección de inputs del jugador.
7. Pausar el juego y detectar si el jugador perdió.

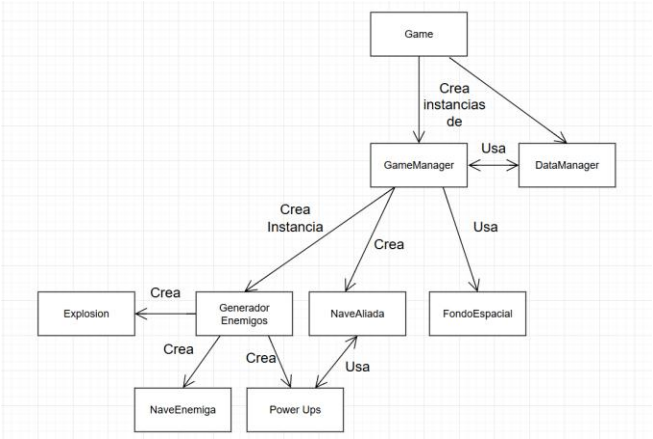
p3. Requerimientos funcionales más relevantes

ID	Requerimiento funcional	Descripción
RF1	Movimiento del jugador	La `NaveAliada` (jugador) puede desplazarse libremente dentro de los límites de la pantalla.
RF2	Disparo del jugador	El jugador puede disparar proyectiles; los power-ups pueden modificar la cadencia o el tipo de arma.
RF3	Aparición de enemigos	Las `NaveEnemiga` se generan en intervalos definidos y se desplazan hacia la `NaveAliada`.
RF4	Colisiones	Se detectan colisiones entre balas, naves y power-ups.
RF5	Power-ups	Al ser recogidos, para activar sus efectos es necesario presionar el click izquierdo.
RF6	Sistema de puntaje	El puntaje aumenta al destruir enemigos.
RF7	Estados del juego	Se manejan los estados: *Menú principal*, *Jugando*, *Pausa* y *Game Over*.
RF8	Efectos visuales	Se generan explosiones y animaciones ante impactos y destrucciones.
RF9	Recolección de estadísticas	El sistema registra métricas de cada partida como puntaje, tiempo de juego, precisión, enemigos destruidos y vidas perdidas

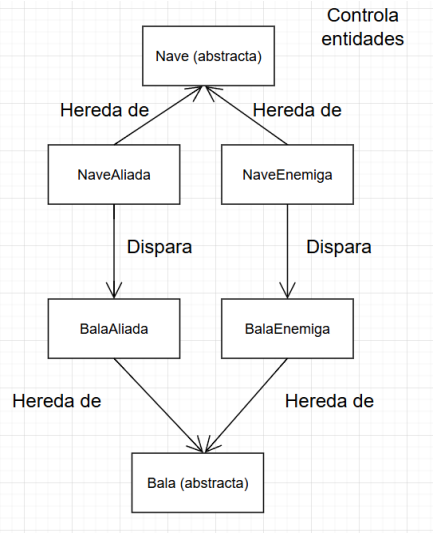
p4. Requerimientos no funcionales

ID	Categoría	Descripción
RNF1	Rendimiento	El juego debe mantener una tasa estable de 60 FPS en equipos de gama media.
RNF2	Mantenibilidad	Cada clase debe tener responsabilidades bien definidas y mínima dependencia entre módulos.
RNF3	Escalabilidad	La estructura del código debe permitir añadir nuevos enemigos o power-ups sin modificar la lógica base.
RNF4	Contoles	Se maneja con el mouse: Click izquierdo para disparar y Click derecho para usar un Power Up. La respuesta debe ser menor a 1 segundo.
RNF5	Fiabilidad	El sistema no debe presentar bloqueos ni errores durante la ejecución.
RNF6	Portabilidad	El código debe ser compatible con la IDE de Processing en Windows, Linux.

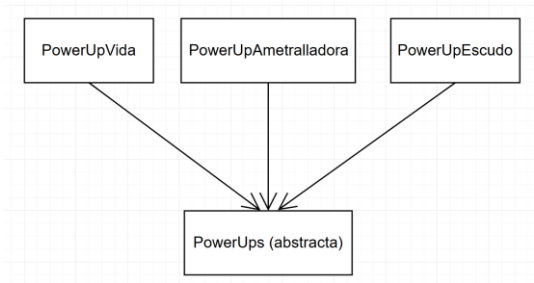
p5. Diagrama Conceptual



El diagrama conceptual muestra la estructura general del sistema, representando las clases principales y sus relaciones. El juego inicia en la clase Game que se encarga de crear la instancia de GameManager, la responsable de coordinar todas las entidades del juego y Data Manager, que administra el guardado de los datos y el registro de puntaje.
Las clases del diagrama instanciadas desde GameManager son FondoEspacial, manejando la animacion del fondo, la Nave Aliada (Controlada por el jugador) y el Generador de Enemigos con el trabajo de controlar todas las interacciones de las NaveEnemigas, incluyendo la generación de ellas y crear ambos Explosiones y Power Ups al momento que alguna se elimina.

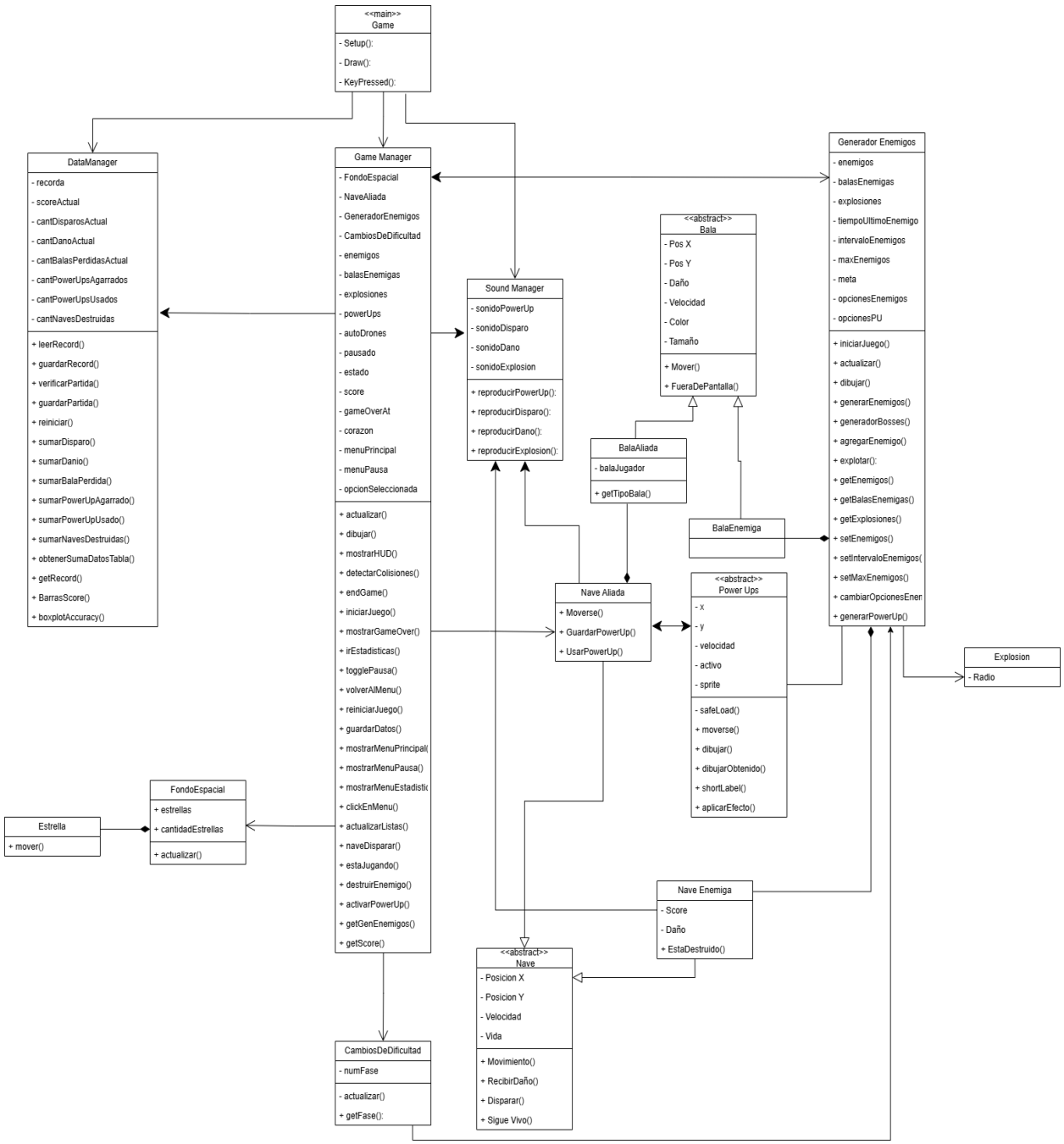


Después, el sistema, cuenta con clases abstractas como Nave que define el comportamiento común entre la NaveAliada (controlada por el jugador) y las NaveEnemiga (controladas automáticamente). Ambas naves pueden generar proyectiles represantados por BalaAliada y BalaEnemiga, derivadas de otra clase abstracta llamada Bala.
Por último, la clase abstracta PowerUps que sirve para la aparición de “poderes” que el usuario puede obtener en determinados momentos del juego. De ella heredan tres tipos específicos: PowerUpVida, PowerUpEscudo, PowerUpAmetralladora, cada uno con un efecto distinto sobre la nave del jugador.



Este diagrama muestra la organización del sistema dejando en evidencia el uso de herencia, composición y encapsulamiento, principios fundamentales de la Programación Orientada a Objetos empleados en el diseño del juego.

p6. Diagrama de clases del modelo



La clase Game es el inicio del programa, está encargada de inicializar el entorno y coordinar el ciclo de ejecución mediante los métodos setup(), draw() y keyPressed(). Desde esta clase se instancian GameManager, Data Manager y Sound Manager.

GameManager que contiene la lógica principal de la partida, gestiona las entidades, colisiones y el guardado de datos mediante métodos como actualizar(), pausar() y guardarDatos().

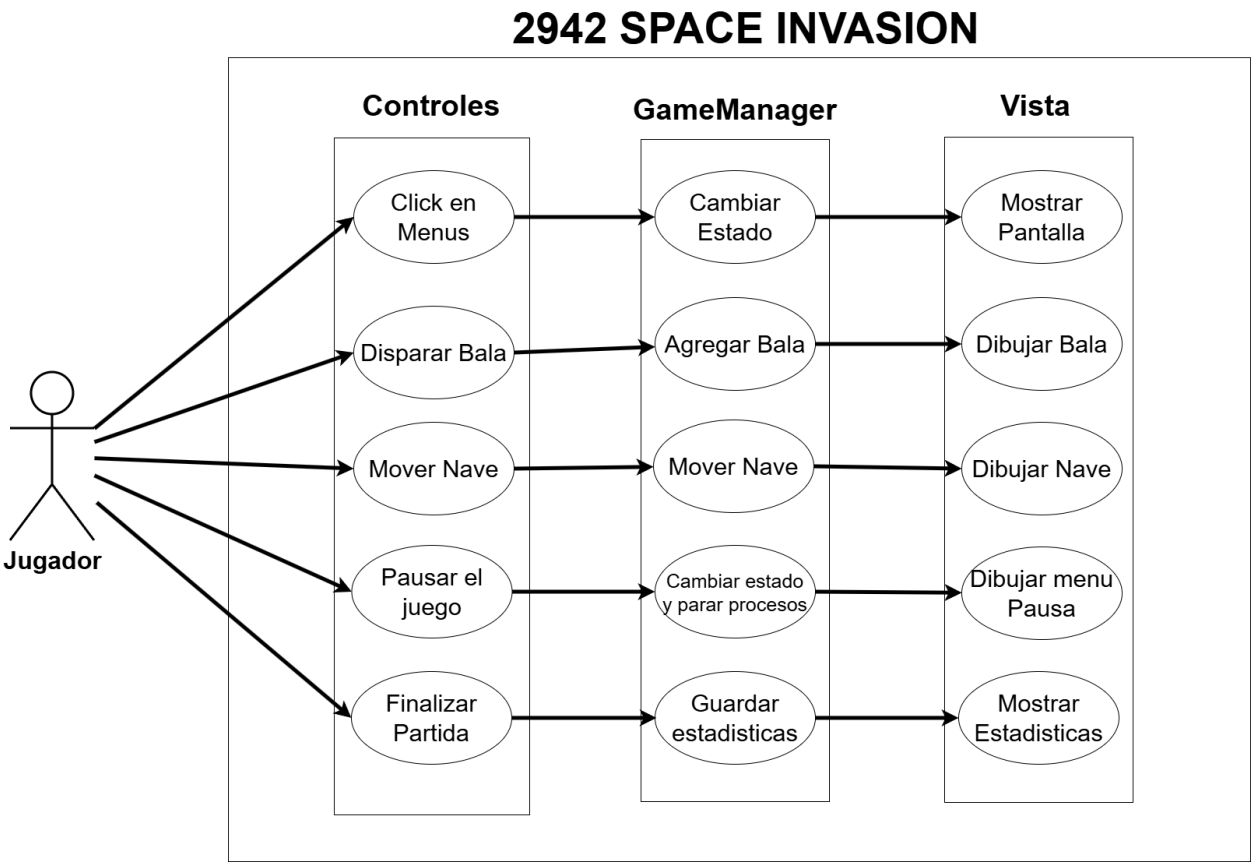
DataManager es el responsable del guardado y análisis del estadísticas del jugador, administrando atributos como cantDisparos, cantPowerUpsAgarrados y cantNavesDestruidas, junto con métodos como leerRecord() y sumarDisparo().

El modelo también cuenta con clases como FondoEspacial, que genera el movimiento de estrellas en el fondo mediante actualizar(), y SoundManager, que se ocupa de reproducir los efectos de sonido a través de métodos como reproducirPowerUp() y reproducirExplosion().
Las entidades del juego se agrupan bajo clases abstractas: Nave, Bala y PowerUp.

Nave define atributos de posición, velocidad y vida, y métodos que representan su comportamiento (moverse(), recibirDaño(), disparar()). De clase Nave, heredan NaveAliada y NaveEnemiga, que agregan comportamientos específicos.
Bala es una clase que explica la lógica de los proyectiles del juego, mientras que PowerUp representa las mejoras temporales que el jugador puede recolectar mientras la partida está en curso.

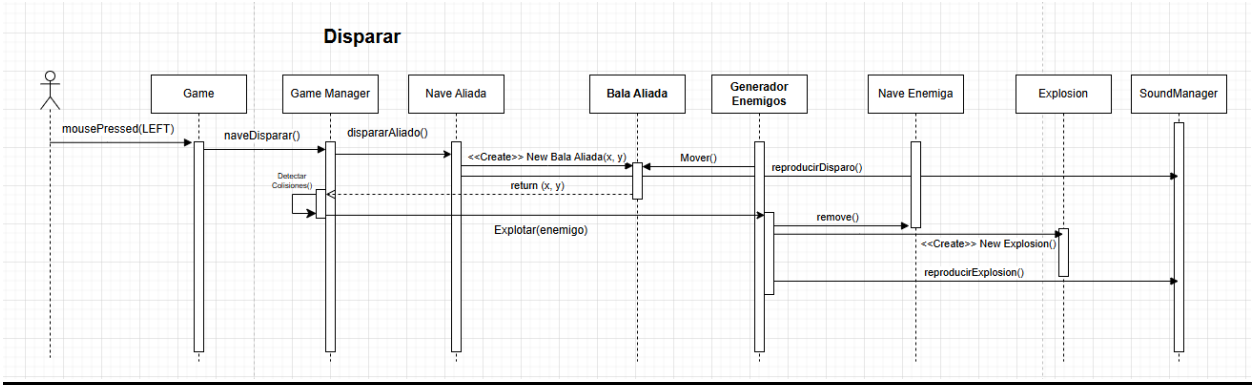
Para terminar con este diagrama hay clases como Explosion, GeneradorEnemigos y CambiosDeDificultad que complementan la jugabilidad controlando efectos visuales, creación de enemigos y aumentando la dificultad a medida que el jugador alcanza una determinada cantidad de puntos.
Este diseño muestra las relaciones de herencia y composición, pero deja en evidencia la aplicación de principios sólidos de la programación orientada a objetos.

p7.Diagrama de la Vista y Controlador



p8. Diagrama de Secuencia

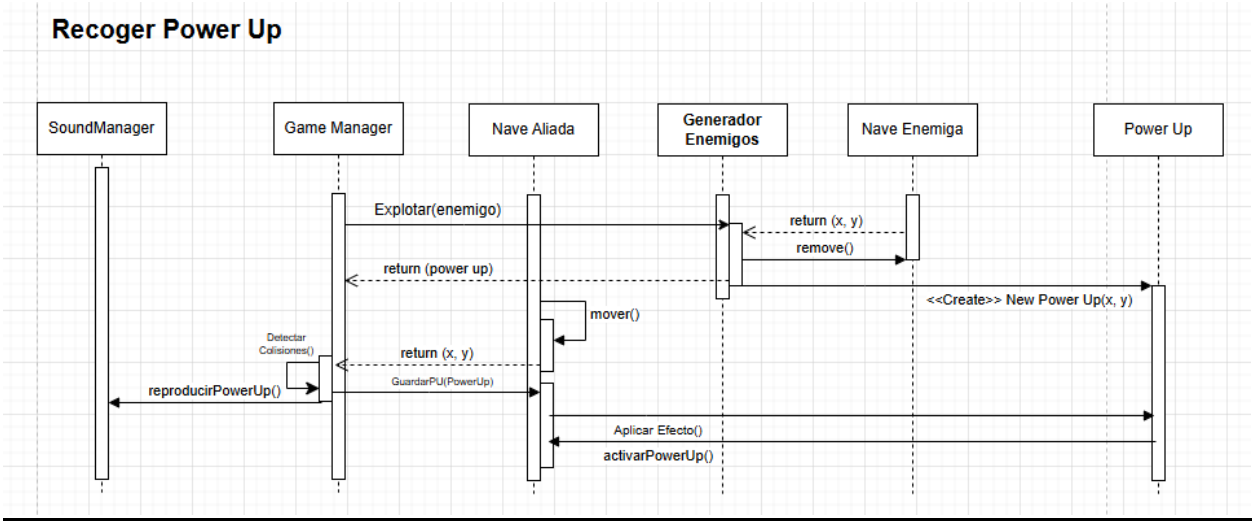
Caso de uso 1: Disparar



El diagrama de secuencia “Disparar” muestra el proceso que se ejecuta cuando el jugador presiona el botón de disparo (mousePressed(LEFT)).

La acción comienza en la clase Game, que detecta la entrada del usuario y envía el mensaje naveDisparar() al GameManager. Este llama al método dispararAliado() de la NaveAliada que crea una nueva instancia de BalaAliada con las coordenadas actuales de la nave.
La bala generada inicia su movimiento mediante el método mover() y es supervisada por el GeneradorEnemigos, que detecta posibles colisiones.
Cuando una colisión ocurre, se ejecuta el método explotar(enemigo), eliminando al enemigo y generando una nueva instancia de Explosion.
Simultáneamente, el SoundManager reproduce los efectos de sonido correspondientes a la acción de disparo (reproducirDisparo()) y a la explosión (reproducirExplosion()).
Este diagrama muestra la colaboración dinámica entre las clases Game, GameManager, NaveAliada, BalaAliada, GeneradorEnemigos, NaveEnemiga, Explosion y SoundManager, mostrando el ciclo completo de un disparo y su resultado en pantalla.

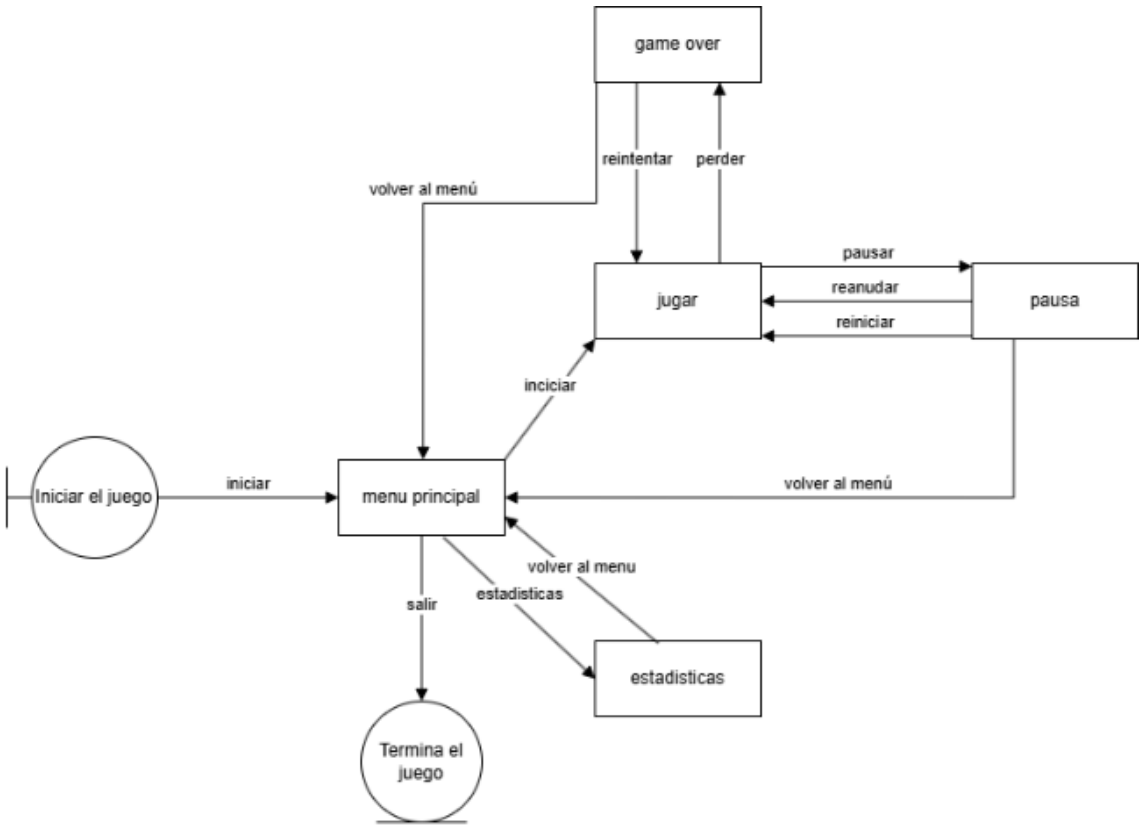
Caso de uso 2: Recoger Power-Up



Este diagrama de secuencia representa el proceso que ocurre cuando la nave aliada colisiona con un objeto de mejora. Esta secuencia comienza cuando el GameManager detecta la colisión entre la NaveAliada y un objeto PowerUp. Una vez que se identifica esa colisión, el SoundManager reproduce el sonido correspondiente (reproducirPowerUp()) y se llama al método guardarTipoPowerUp() para registrar el evento.
Luego, la clase NaveAliada activa el efecto del objeto recolectado mediante el método activarPowerUp(), que invoca el método aplicarEfecto() dentro del PowerUp. Dependiendo del tipo de mejora (vida, escudo o ametralladora) se modifican, de manera temporal, las propiedades de la nave.

Durante este proceso, intervienen también el GeneradorEnemigos y el GameManager, que continúan controlando las colisiones y actualizando el estado general del juego.
Este diagrama enseña la interacción coordinada entre GameManager, NaveAliada, PowerUp, SoundManager, GeneradorEnemigos y NaveEnemiga, mostrando cómo se maneja la recolección de un objeto y la aplicación de su efecto en tiempo real.

p9. Diagrama de Navegacion GUI

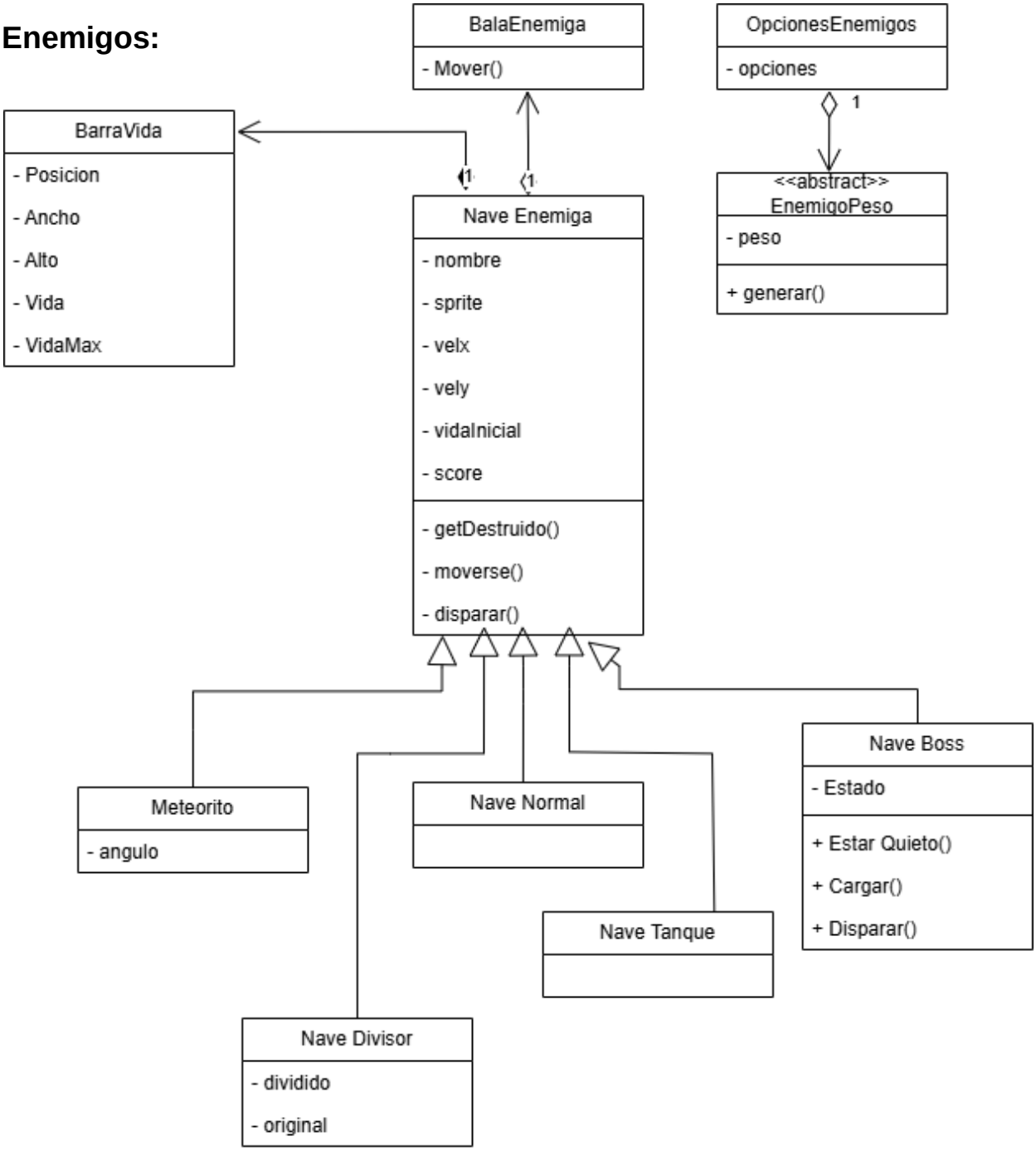


El diagrama de navegación representa el flujo entre las diferentes pantallas del juego.
Desde el Menú Principal el jugador puede iniciar la partida, que avanza al estado de Juego en curso.
Durante la partida se puede acceder a Pausa, y al finalizar se muestra la pantalla de Game Over, desde la cual se puede volver al menú o acceder al módulo de estadísticas.

Contenidos opcionales:

Diagrama de clases de un módulo o subsistema relevante:

Enemigos:



Power Ups:

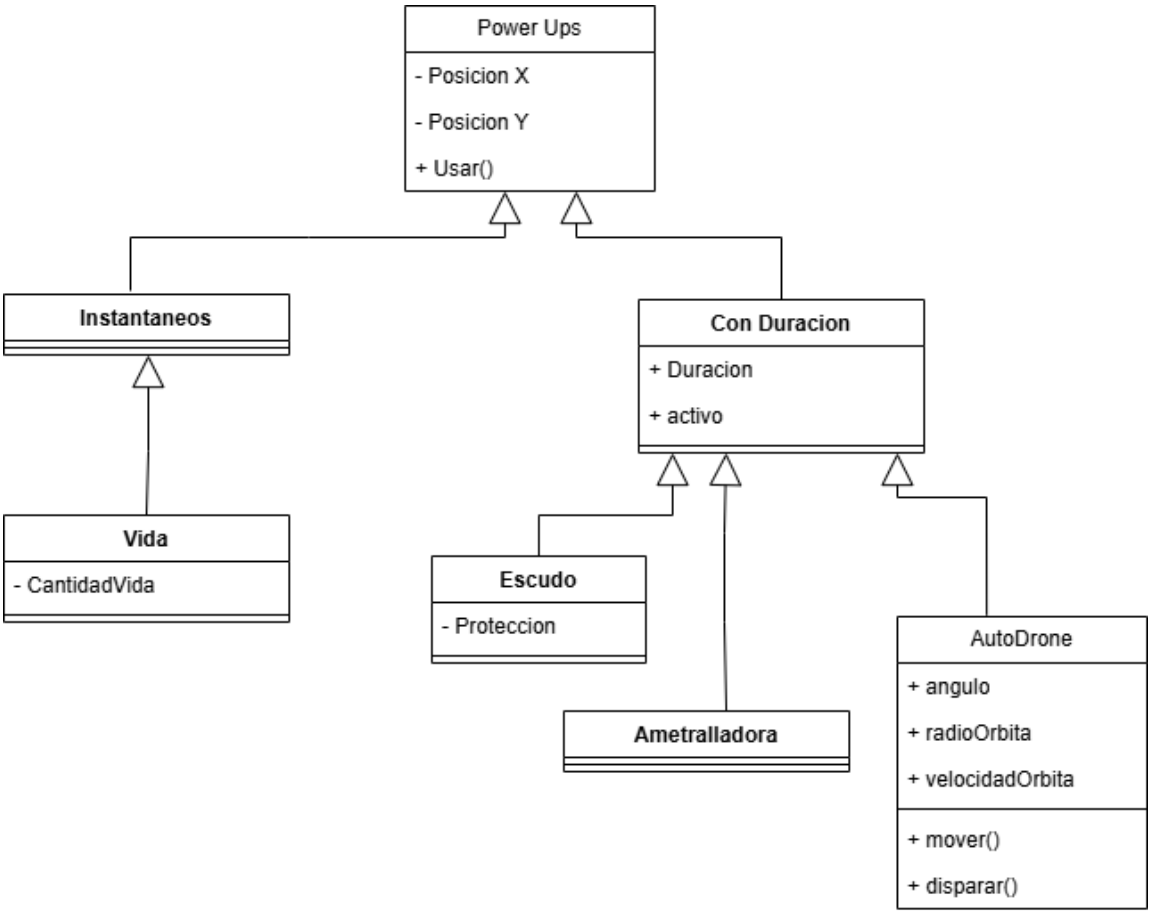
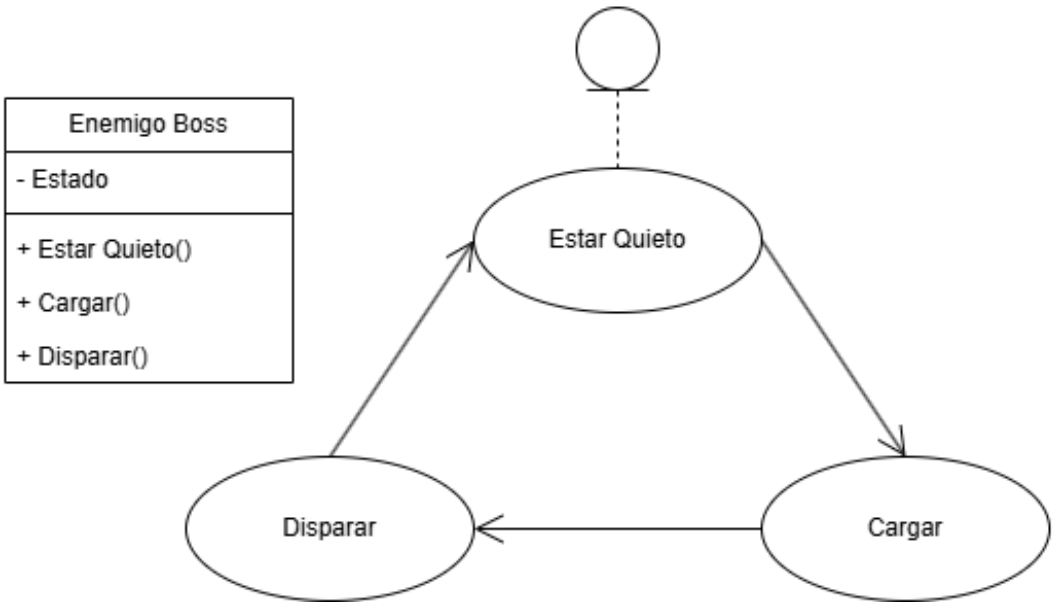


Diagrama de estados:

Enemigo Boss, Máquina de Estados: